Root Finding in C: A Beginner's Guide

Finding the roots of an equation means solving for values of x where $f(x) = 0$. Many equations cannot be solved algebraically, so we use numerical methods. This tutorial explains three common numerical root-finding techniques and their implementation in C.

Understanding the Problem

Given a function $f(x)$, we seek values of x such that:

$$f(x) = 0$$

We will implement three methods:

- **Bisection Method**: A simple, reliable method based on interval halving.
- **Newton's Method**: A faster method that uses derivatives.
- **Secant Method**: Similar to Newton's method but does not require derivatives.

Method 1: Bisection Method

The **Bisection Method** is based on the Intermediate Value Theorem: if $f(a)$ and $f(b)$ have opposite signs, then there is at least one root in $[a, b]$.

Steps:

1. Check if $f(a)$ and $f(b)$ have opposite signs.
2. Compute the midpoint $c = (a + b)/2$.
3. Replace a or b with c based on where the sign change occurs.
4. Repeat until the interval is small enough.

C Implementation:

```
# include <stdio.h>
# include <math.h>

// Define the function
double f(double x) {
    return pow(x,3) + x - 1; // Example: f(x) = x^3 + x - 1
}

// Bisection Method
double bisection(double a, double b, double tol) {
    if (f(a) * f(b) >= 0) {
        printf("Invalid interval: f(a) and f(b) must have opposite signs.\n");
        return -1;
```

```
    }
    double c;
    while ((b - a) / 2 > tol) {
        c = (a + b) / 2;
        if (f(c) == 0.0)
            break;
        else if (f(c) * f(a) < 0)
            b = c;
        else
            a = c;
    }
    return c;
}
```

Method 2: Newton's Method

Newton's Method uses the tangent line to approximate the root. Newton's Method, also called the **Newton–Raphson** Method, usually converges much faster than the linearly convergent methods we have seen previously.

The formula is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Pseudo-code:

```
Newton-Raphson Method (x0, tol, max_iter)

Given an initial guess x0
For i = 0 to max_iter:
    Compute f(x) and f'(x)
    if f'(x) = 0, stop, return failure
    Compute x_new = x_old - f(x_old) / f'(x_old)
    if |x_new - x_old| < TOL, stop, return x_new
    Update x_old = x_new
End For
Return x_new
```

C Implementation:

```
// Define derivative of f(x)
double df(double x) {
    return 3 * x*x + 1; // Example: f'(x) = 3x^2 + 1 for f(x) = x^3 + x - 1
}
```

```
// Newton's Method
double newton(double x0, double tol, int max_iter) {
    double x = x0;
    for (int i = 0; i < max_iter; i++) {
        double fx = f(x);
        double dfx = df(x);
        if (dfx == 0) {
            printf("Derivative is zero, Newton's method fails.\n");
            return -1;
        }
        double x_new = x - fx / dfx;
        if (fabs(x_new - x) < tol)
            return x_new;
        x = x_new;
    }
    return x;
}
```

Method 3: Secant Method

The **Secant Method** approximates the derivative using two previous points, avoiding the need for an explicit derivative function. Geometrically, the tangent line is replaced with a line through the two last known guesses. The intersection point of the “secant line" is the new guess.

Formula:

$$x_{n+1} = x_n - f(x_n) \times \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Pseudo-code:

```
Secant Method (x0, x1, tol, max_iter)

Given two initial guesses x0 and x1
For i = 0 to max_iter:
    Compute f(x0) and f(x1)
    if f(x1) - f(x0) = 0, stop, return failure
    Compute x_new = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
    if |x_new - x1| < TOL, stop, return x_new
    Update x0 = x1, x1 = x_new
End For
Return x_new
```

C Implementation:

```
// Secant Method
double secant(double x0, double x1, double tol, int max_iter) {
    for (int i = 0; i < max_iter; i++) {
        double fx0 = f(x0);
        double fx1 = f(x1);
        if (fx1 - fx0 == 0) {
            printf("Division by zero in Secant method.\n");
            return -1;
        }
        double x_new = x1 - fx1 * (x1 - x0) / (fx1 - fx0);
        if (fabs(x_new - x1) < tol)
            return x_new;
        x0 = x1;
        x1 = x_new;
    }
    return x1;
}
```

Testing the Methods

Create a `main` function to test all three methods:

```
int main() {
    double root_bisect = bisection(0, 3, 0.0001);
    printf("Root (Bisection): %lf\n", root_bisect);

    double root_newton = newton(3, 0.0001, 20);
    printf("Root (Newton): %lf\n", root_newton);

    double root_secant = secant(0, 3, 0.0001, 20);
    printf("Root (Secant): %lf\n", root_secant);

    return 0;
}
```

Problem Statement

A company produces and sells a product, and its **cost function** and **revenue function** are given as:

$$C(x) = 500 + 10x$$

$$R(x) = 20x$$

where:

- $C(x)$ is the **total cost** for producing x units
- $R(x)$ is the **total revenue** from selling x units
- x represents the number of units produced and sold

The **breakeven point** occurs when the company neither makes a profit nor a loss, meaning:

$$C(x) = R(x)$$

Find x using numerical root-finding methods!

Define the Function
Rearrange the equation to form a root-finding problem:

$$f(x) = R(x) - C(x) = 20x - (500 + 10x) = 10x - 500$$

So, we need to find x where:

$$f(x) = 0$$

Using the Bisection Method

```
# include <stdio.h>

// Define the function for breakeven point
double f(double x) {
    return 10 * x - 500; // f(x) = 10x - 500
}
```

Breakeven point: 50.00 units

This means that **at 50 units**, the company covers its costs exactly, and from that point onward, it starts making a profit.

Key Takeaways

1. **Bisection is simple and reliable but slow.**
2. **Newton's method is fast but needs a derivative and a good initial guess.**
3. **The Secant method is like Newton's method but does not need derivatives.**

```
Start coding or generate with AI.
```